5. Cyclic Codes

Cyclic codes are widely used linear block codes. We define them as such linear block codes in which the rotated version $\mathbf{c}^s$ of every valid code word $\mathbf{c}$ is also a valid code word:

$$\boldsymbol{c} = [c_0 \quad c_1 \quad \dots \quad c_{N-2} \quad c_{N-1}], \qquad \boldsymbol{c}^s = [c_{N-1} \quad c_0 \quad c_1 \quad \dots \quad c_{N-2}]$$

In cyclic codes, there can always be found a valid code word of the form $[g_0\ g_1 \dots g_{N-K}\ 0 \dots 0]$, so the generator matrix of these codes can always be written in a band matrix form:

$$\boldsymbol{G} = \begin{bmatrix} g_0 & g_1 & \dots & g_{N-K} & 0 & \dots & \dots & 0 \\ 0 & g_0 & g_1 & & \ddots & & & \vdots \\ 0 & & g_0 & g_1 & & \ddots & & \vdots \\ \vdots & & & \ddots & \ddots & & \ddots & 0 \\ 0 & \dots & \dots & 0 & g_0 & g_1 & \dots & g_{N-K} \end{bmatrix}$$

Example 5.1
An example for a simple (4,2) cyclic code:
$\boldsymbol{c}_0 = [1\ 0\ 1\ 0]$
$\boldsymbol{c}_1 = [0\ 1\ 0\ 1]$
$\boldsymbol{c}_2 = [0\ 0\ 0\ 0]$
$\boldsymbol{c}_3 = [1\ 1\ 1\ 1]$

It can be seen, that as a result of the band matrix structure, the vector components of **c** = **mG** are the convolution sum of series (m_0, m_1, …, m_{K-1}) and (g_0, g_1, …, g_{N-K}). (For that, note that series (g_0, g_1, …, g_{N-K}) also appears vertically in the matrix rows, but in a reversed index order.)

Using this property, we introduce the bit position operator x, where the exponent of x marks the position of the symbol, and instead of vectors, we will work with polynomials of x. Numbers in the exponents are integers, while the coefficients are elements of GF[2]. Since the convolution product of the polynomial coefficient sequences consists of coefficients of polynomial multiplication, the application of the x operator turns the matrix-vector multiplication into polynomial multiplication. The motivation for this transform is that binary polynomial multiplication and division can be implemented in hardware even simpler than matrix-vector multiplication. In summary:

$$\boldsymbol{c} = [c_0 \quad c_1 \quad c_2 \quad \dots \quad c_{N-1}] \xrightarrow{x} c(x) = c_0 + c_1 x + c_2 x^2 + \dots + c_{N-1} x^{N-1}$$
$$\boldsymbol{c} = \boldsymbol{mG} \xrightarrow{x} m(x)g(x)$$

where $g(x) = g_0 + g_1 x + g_2 x^2 + \dots + g_{N-K} x^{N-K}$ is the *generator polynomial*. We write the members of the polynomials in an increasing order of the exponent x.

It is easy to show that applying the normal arithmetic polynomial division with a remainder

$$x \cdot c(x) = c^s(x) \bmod (1 + x^N)$$

This means that multiplication by x, the bit position operator, is equivalent to rotation (cyclical shifting), but only modulo $1 + x^N$. So we introduce the ring $GF(2)[x] \mid 1 + x^N$, which is commutative and has identity elements, and whose elements are polynomials of degrees less than N. In this ring we define two operations, polynomial addition and multiplication (denoted by + and ·) in the usual way. In cases when multiplication would "lead outside" of the ring, the result is divided by the basic polynomial ($1 + x^N$) and the remainder is kept.

Later on, we will utilize the easily proven fact that the remainder for the sum of two polynomials is the sum of the individual remainders.

5.1. Construction of Cyclic Codes

Choose such a $g(x)$ that $h(x) \cdot g(x) = 1 + x^N$ is satisfied, and generate the valid code words, based on $c(x) = m(x)g(x)$. Then for the valid code words in the polynomial ring:

$$s(x) = c(x) \cdot h(x) = m(x) \cdot g(x) \cdot h(x) = 0 \bmod x^N + 1$$

Using a reasoning similar to the one is Section 4.3, it can be proved that the statements of the parity-check theorem are also true for the generated cyclic codes. These codes can be directly implemented by using polynomial multiplier circuits (both at the sender and at the receiver) which consist of a shift register and a combination logic containing the polynomial coefficients (for detailed information see Figure 6.1 in the convolutional codes section). The error correction for correctable errors can be executed by using the syndrome table. However, in decoding, it causes some trouble that the code is not systematic, thus the message estimate cannot be computed from the corrected code word.

Systematic Cyclic Codes

To make the code systematic, we start again with $h(x) \cdot g(x) = 1 + x^N$, but the valid code words are generated differently, in the following way:

$$c(x) = x^r m(x) + d(x), \text{ where}$$
$$d(x) = x^r m(x) \mid g(x)$$

As $deg(d(x)) < N - K$, the code is indeed systematic, thus its first K positions are given only by $x^r m(x)$, and the last N – K by $d(x)$. At the receiver, the syndrome is given by the remainder modulo $g(x)$:

$$s(x) = c'(x) \mid g(x)$$

Then, for the valid code words

$$s(x) = \left(x^r m(x) + d(x)\right) \mid g(x) = x^r m(x) \mid g(x) + x^r m(x) \mid g(x) = 0$$

as the polynomial coefficients are elements of GF[2] (so 1 + 1 = 0).

The code generated is also cyclic, as it is equivalent to a code created by $g(x)$ via polynomial multiplication, as every code word is a multiple of $g(x)$. To see this, introduce $b(x)$ in the following way:

$$x^r m(x) = b(x)g(x) + x^r m(x) \mid g(x)$$

Then the systematic code word $c(x)$ can be written as

$$c(x) = b(x)g(x) + x^r m(x) \mid g(x) + x^r m(x) \mid g(x) = b(x)g(x)$$

so indeed, every valid code word is a multiple of $g(x)$.

It can also be shown, that condition $h(x) \cdot g(x) = 1 + x^N$ is not just sufficient, but also necessary for the generated code word to be cyclic. Another necessary condition is that in $g(x)$, the coefficients of smallest and highest degree are both 1.

Implementing Systematic Cyclic Codes with Polynomial Divider Circuit

As already mentioned, for the sender's cyclic code generation, and for the receiver's syndrome calculation, the same device – a divider by $g(x)$ circuit – is sufficient. The main idea behind the circuit is that for arbitrary $w(x)$:

$$w(x) \mid g(x) = w_0 \mid g(x) + w_1 x \mid g(x) + w_2 x^2 \mid g(x) + \cdots + w_{N-1} x^{N-1} \mid g(x)$$

So the remainders are calculated member wise. The structure of such circuit at the sender's side can be seen in Figure 5.1

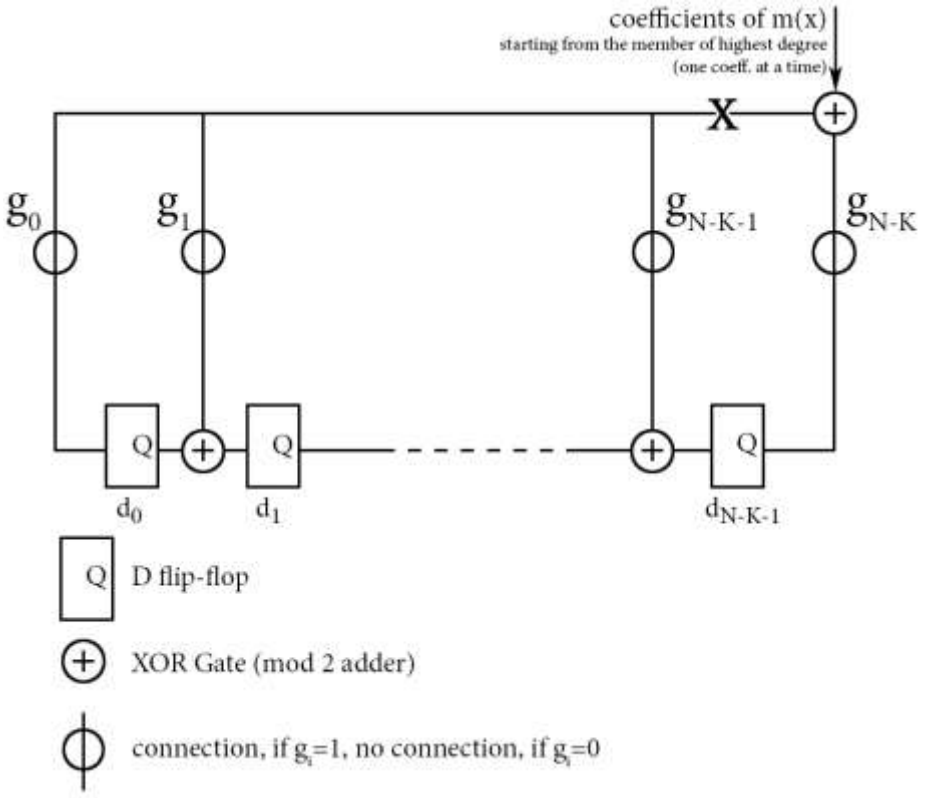


Figure 5.1: *Structure of a binary polynomial divider. The feedback link marked by an X is referred to in Example 5.1*

The content of the D latches at the beginning is 0. After K cycles, the latches store the coefficients (d_0, …, d_{N-K-1}) of the remainder of the division by *g(x)*. To produce a systematic code, the divider circuit is used in two modes:

1. First, for K cycles we load the coefficients of *m(x)*, while at the same time, we send these to the channel too (so the code is systematic). After K "ticks", the remainder of *d(x)* will be in the latches.
2. We disable the feedback marked by the X in Figure 5.1, and instead of the output of the XOR gate, the feedback line gets 0's. Then, for N – K clock cycles, we send the content of the rightmost latch to the channel, that is, we empty the shift register. As a "side effect", the circuit gets back to its initial state, and will be ready for producing the next code word.

The **decoder** executes the division for N cycles with the same circuit[14], then the resulting syndrome is used to address the syndrome table (ROM), in order to identify the error vector estimate, which is then added to the code word received. For proper scheduling, the incoming code word has to be buffered for N cycles in a shift register.

Example 5.1
Generating a cyclic code and parity-checking for N = 7
Factoring the initial polynomial we get:

$$1 + x^7 = (1 + x)(1 + x^2 + x^3)(1 + x + x^3)$$

[14] The circuit also automatically executes multiplication by x^r, so we do not really generate the syndrome in the same way as originally planned. However, it can be easily shown, that the statements of the parity-check theorem still hold, and the syndrome table is usable.

Let

$g(x) = 1 + x^2 + x^3$, *which means K = 4, r = 3*

$m(x) = 1 + x + x^3$

Then

$c(x) = x^3 m(x) + x^3 m(x) \mid g(x) = x^3 + x^4 + x^6 + x^2 = [0\ \ 0\ \ 1\ \ 1\ \ 1\ \ 0\ \ 1]$

Achieving the same with the polynomial divider:

Tick	$d_0 d_1 d_2$	*Point "X"*	*Input*
0	*0 0 0*	*1*	*1*
1	*1 0 1*	*1*	*0*
2	*1 1 1*	*0*	*1*
3	*0 1 1*	*0*	*1*
4	*0 0 1 = x^2 is the remainder*		

From this, the code word is: c(x) = $x^2 + x^3 + x^4 + x^6$, that is checked at the receiver:

Tick	$d_0 d_1 d_2$	*Point "X"*	*Input*
0	*0 0 0*	*1*	*1*
1	*1 0 1*	*1*	*0*
2	*1 1 1*	*0*	*1*
3	*0 1 1*	*0*	*1*
4	*0 0 1*	*0*	*1*
5	*0 0 0*	*0*	*1*
6	*0 0 0*	*0*	*0*
7	*0 0 0 the remainder is 0 (the code word is valid)*		

By generating all 16 valid code words of the codes it can be seen, that

a) the code is indeed cyclic

b) code distance is 3, so we can correct 1 error

In a similar way, it can be computed that the syndrome for the wrong code word c'(x) = $x^2 + x^3 + x^4$, affected by the 1 weighted e(x) = x^6 error vector, is x^2. We get the same syndrome if we compute the remainder for x^6, as the syndrome depends only on the error. Note that the previous code is a (7, 4) Hamming code, as it is perfect and can correct 1 error. Hamming codes form a subclass of cyclic codes.

Example 5.2

(K+1, K) parity code, as a linear block code, and as a cyclic code

Linear block code:

The redundant part is 1 bit, which is the mod 2 sum of the digits of the message:

$$\boldsymbol{G} = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & \dots & 0 \\ 1 & 0 & 1 & 0 & 0 & & \\ 1 & 0 & 0 & 1 & 0 & \dots & 0 \\ \vdots & & & & 1 & & \\ & & & & & \ddots & \\ 0 & 0 & & \dots & & 1 & 1 \end{bmatrix}$$

Implementation: XOR gate with K inputs, addition in parallel time

Cyclic code:

*By using addition for the rows of the previous **G** matrix, a cyclic code is created easily:*

$$\boldsymbol{G} = \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & \dots & 0 \\ 0 & 1 & 1 & 0 & 0 & & \\ 0 & 0 & 1 & 1 & 0 & \dots & 0 \\ \vdots & & & 1 & 1 & & \\ & & & & \ddots & \ddots & \\ 0 & 0 & & \dots & & 1 & 1 \end{bmatrix}$$

From this g(x) = 1 + x. Indeed, this g(x) is a divisor of 1 + x^N for every N.
Implementation: The polynomial divider, based on Figure 5.1, is a single D-FF with feedback from a XOR gate. This executes the same addition operation in bit serial mode (in K ticks, by storing the partial result) as a XOR gate with K inputs in transient time.

Cyclic Codes Used in Practice

Only a part of cyclic codes can be designed (in a way not detailed here) for given N, K and d_{min} parameters. The most important are:

- Hamming codes
- BCH codes (Bose–Chaudhuri–Hocquenghem). As the number of correctable errors increase, so does the complexity of encoders and decoders. For example, the code generated by the polynomial 3551[15] has parameters (31, 21), and it can correct 2 errors.

CRC Codes

Cyclic Redundancy Check codes are able to only detect errors, therefore, they are typically used together with some kind of error correcting coding technology. CRC codes are special, shortened cyclic codes, that create short (e.g. r = 16) check sums for big block sizes (e.g. K = 32000). If anything changes in the content of the block or in the check sum, it has a great probability that the CRC sum calculated at the receiver will not match the one at the end of the block sent. CRC codes and the syndrome can be generated by polynomial division just like cyclic codes. The main steps of using CRC:

1. After source encoding, but before error correcting channel encoding, we generate CRC for each blocks and place them to their ends.
2. Divide the message into blocks again, based on the size of the error correcting code. Generate the error correcting code.
3. At the receiver, after parity-check/correction, for every CRC block the CRC sum is checked. If the received and the calculated one differ, then there is an error in the block, so the error correction was not successful. So we send a notification of the detected error and request the resending of the CRC block.

By using CRC, it is possible to detect the cases of block decoding errors at the receiver (correcting into another valid code word due to too many errors; the error changing the message into another valid code word) with relatively small loss in entropy rate. This is true, because block sizes for error correction are typically smaller than in case of CRC, so one CRC block covers more error correcting blocks.

Naturally, using CRC is unsuccessful, if an error occurs in the CRC block itself, but we get no CRC errors. This can happen in the two following cases:

1. The checksum did not change, but the block changed in a way that the result provides the same sum as before.
2. Both the block and the sum changes, but in such a way, that the new sum will match the new block.

[15] If g(x) is given in this way, by octal (base-8) numbers, then we can get the coefficients of g(x) by transforming every digit into binary (base-2) form. In this case, every digit encodes 3 coefficients, e.g. 4 → 100. On the left side, stands the coeff. of highest degree. An example: $g(x) = 57 = x^5 + x^3 + x^2 + x + 1$.

Therefore, the ability to detect errors does not depend only on the number of errors, but on their positions (pattern) too. The quality related properties of CRC codes are the following:

- **Error detection capability**: this can be determined by exact analysis of the specific code. It can be shown, that the undetected error probability approaches 2^{-N} as K increases.
- **Error pattern coverage**: the probability that a randomly chosen CRC-block and a check-sum does not match (it is not a valid CRC block). As the number of such pairs that *do not* produce CRC errors is 2^{N-K}, the coverage is $\lambda = 1 - \frac{1}{2^{N-K}}$, that depends only on the degree of *g(x)*.
- **Burst error detection capability**: Due to some physical effects on the channel (e.g. smudges on a disk, power outages etc.) errors in most cases tend to appear not in a random distribution, but close to each other, in so-called *bursts*[16]. For CRC codes, if the length of a burst is *b*, then the code detects every error burst, if $b \leq N - K$, and $1 - \frac{1}{2^{N-K}}$ part of them if $b > N - K + 1$.

5.2. Reed-Solomon Codes

Reed-Solomon codes are non-binary cyclic codes defined over a finite field GF[q], where q is either a prime number or a power of a prime number p. It can be shown that a finite field always has a *primitive* element a such that $a^{q-1} = 1$ and all the other powers of a are different, which means that the set of these powers is identical to the set of all elements in GF[q].

A typical setup of Reed-Solomon codes in information technology is when we choose $q = 2^8$ such that we have field of 256 elements and an element (a symbol of the code) will be equivalent to a byte. This yields an efficient hardware and software implementation.
For an (N, K) linear block code we choose q such that $q - 1 > N - 1$ and we define the generator matrix **G** of the code as follows.

$$\mathbf{G} = \begin{bmatrix} 1 & 1 & 1 & 1 & \cdots & 1 \\ 1 & a & a^2 & a^3 & \cdots & a^{N-1} \\ 1 & a^2 & a^4 & a^6 & \cdots & a^{2(N-1)} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & a^{K-1} & a^{2(K-1)} & a^{3(K-1)} & \cdots & a^{(K-1)(N-1)} \end{bmatrix}$$

When we multiply the message vector $\mathbf{m} = [m_0 \quad m_1 \quad m_2 \quad \cdots \quad m_{K-1}]$ with this generator matrix we get the following components of the code vector **c**:

$$\begin{gathered} c_0 = m_0 + m_1 + m_2 + \cdots + m_{K-1} \\ c_1 = m_0 + m_1 a + m_2 a^2 + \cdots + m_{K-1} a^{K-1} \\ c_2 = m_0 + m_1 a^2 + m_2 a^4 + \cdots + m_{K-1} a^{2(K-1)} \\ \text{etc.} \end{gathered}$$

We now make use of the fact that the above components are indeed the values of the polynomial *m(x)* at $a^0, a^1, a^2, \cdots, a^{N-1}$. Note that $a^0, a^1, a^2, \cdots, a^{N-1}$ are all different because a is a primitive element of the field and $N < q$. We also know from the Fundamental Theorem of Algebra. that an *n*-degree polynomial cannot have more than *n* different roots. From these it follows that there will be at least $N - (K - 1) = r + 1$ non-zero components in any valid code vector of a Reed-Solomon code, i.e. the weight of any code word is at least $r + 1$. This in turn implies, by the power of the code distance theorem, that the code distance is $r + 1$, so the Reed-Solomon

[16] For block codes we define a burst as the distance in the positions of the first and last wrong channel symbol in a code word block of length N, regardless of any errors in between. So even if only the first and last symbols are wrong, the length of the burst is still N.

codes are maximum distance separable (MDS) codes. In the specific case when $K = 1$, the Reed-Solomon code turns into the $(N, 1)$ repetition code.

The code can be made systematic by column-row transforms of the generator matrix.

Problem. Assume that $q = 7, a = 3, N = 6, K = 3$. Compute the systematic generator matrix of the (6, 3) Reed-Solomon code. *Do not forget that all calculations must be performed in the finite field, i.e. modulo 7.*

Erasure error correction with Reed-Solomon codes

If we use a channel with erasures, we may make the assumption that the position of all errors are known, so we need to correct only erasure type errors. According to the above, the code distance of the code is $r + 1$, therefore we can correct *r* erasures. If we have r erasures, then we can use the rest K (non-erased) received channel symbols to construct a linear equation system of K equations to determine the K components of the original message vector $\mathbf{m} = [m_0 \quad m_1 \quad m_2 \quad \cdots \quad m_{K-1}]$.

Problem. Assume that with a (6, 3) Reed-Solomon code the received code vector is $\mathbf{c}' = [5 \quad 1 \quad E \quad E \quad 2 \quad E]$ and the systematic generator matrix is

$$\mathbf{G} = \begin{bmatrix} 6 & 1 & 3 & 1 & 0 & 0 \\ 3 & 3 & 6 & 0 & 1 & 0 \\ 6 & 4 & 6 & 0 & 0 & 1 \end{bmatrix}$$

Find the original (corrected) message.

Correction of errors with an unknown location

The correction of general (not erasure type) errors can be performed in the usual way using the parity check matrix computed from G, and a syndrome table in the same way as shown for the Hamming-codes.

Problem. Compute the parity check matrix and (a part of) the syndrome table for the generator matrix above. Correct a single error with the code.

5.3. Interleaving

Interleaving is a method widely used in telecommunications for various purposes, here we will apply it for error burst control.

Using Hardware Redundancy

An obvious defence against error bursts is dividing the to-be-encoded channel symbol stream into two parts, into odd and even time intervals, and then execute the channel encoding in par-allel (maybe even by using different parameters, or encoding techniques). The output pairs of these two encoders are then recombined (interleaved) and transmitted. At the receiver, the words received are split into parts and processed separately with the proper decoders. Then their outputs are arranged (de-interleaved) into the original message sequence.

This process requires 2 encoder/decoder pairs (interleave of 2), but

- These run on half of the channel's clock rate, which is useful in case of fast channels and slow coders (complex algorithms).
- An error burst of length *b* appears on the two streams as two error blocks of length *b*/2, as a result of interleaving. So the communication system needs to handle bursts of half the size than as usual.

Naturally, a similar approach can be utilized for more than double redundancy, to apply n-fold hardware redundancy, which results in decreasing the error burst sizes to 1/n.

Block Interleaver

This solution does not require the usage of multiple coders. The symbol stream encoded by channel coding is used to fill a table of size D×D, in *column-wise* order. When the table is full,

the data is read in a *row-wise* order and sent to the channel. At the recipient, a similar table is loaded in a row-wise order, and when it is full, then the columns are read in their order. It can be seen, that if an error burst emerges on the channel, then up to a specific size ($D^2 - D$), after block de-interleaving at the recipient, it appears in D different blocks, broken into D parts.

A drawback is that filling the table requires D^2 time segments to pass, but this delay can be compensated by buffering.

Application: Storing Music with Reed-Solomon Code

In coding of audio CDs to protect against possible error bursts (e.g. scratches), the block interleaving technique is used. The audio is sampled at 44 KHz, on two channels, and every value recorded is quantized into a 2 byte integer. This way 4 bytes represent one sample. The bytes are written into a 24×24 table *column-by-column (*6 samples per column, 144 in total). Then an error correcting code is used to extend the *rows* of the table, to have 28 columns, and afterwards to extend the *columns* in order to have a 28×28 table. This table is then written onto the disk *row-by-row*. The coding used is a (28, 24) Reed-Solomon code with code distance of 5 over GF[256]. This method splits the error bursts into their one twenty-eighth.

At the receiver, in every row of the table, 1-errors are corrected with the error correcting code. The code could also correct 2-errors, but in those cases the two errors are transformed into an erasure error. For more than 2 errors, the whole row is considered as an erasure error. After the rows, the columns are corrected, where only erasure errors can appear, from which 4 could be corrected, as the code is MDS. Yet the code word is discarded and the missing value is estimated based on its context if more than 2 errors are present.

5.4. Error Rates of Block Codes

The following statements are true for all error correcting block codes. Suppose that we use a binary symmetric channel with error probability p in our system, the code can correct t_c errors and we correct all correctable errors. Then the probability of block decoding error (correcting into another word) at the recipient is

$$P_{bde} = 1 - P_{good\ correction} = 1 - \sum_{i=0}^{t_c} \binom{N}{i} p^i (1-p)^{N-1} \xrightarrow[N \gg 1]{} \approx 1 - e^{-Np} \sum_{i=0}^{t_c} \frac{(Np)^i}{i!}$$

So this probability can be approached as a function of Np and t_c, where Np is the expected number of errors inside the blocks. The question is that for a usable communication system, how much bigger the number of correctable errors should be than that of the expected errors. To answer this, we should take a look at function P_{bde} with regard to t_c/Np. If we require that $P_{bde} > 10^{-10}$(which is quite a weak constraint), then for Np = 2 we get that the code should correct about 8 times more errors. In general, t_c should be not just somewhat bigger than Np, but it should be a greater multiple of Np, in order to have a proper and usable system.

In Section 1.2 we defined our two main goals in information transmission: utilizing channel capacity as much as possible (good entropy rate), and providing the smallest achievable block decoding error probability. When selecting code parameters and coding techniques, we can try two ways to fulfil these:

- Choose N to be small, to have a small Np (fewer errors to be corrected). Unfortunately, for the known block codes, K values close to N are available only for big N values (e.g. for BCH codes correcting 2 errors, if N = 15, then R = K/N = 0.46, and if N = 127, R = 0.89). So it is difficult to achieve good entropy rate in this way.
- Choose N to be large, so by selecting a K small enough we can have enough redundancy for being able to correct the required number of errors. However, this way we also end up with a low entropy rate (K/N).

Therefore, it can be easily seen, that it is challenging to achieve both goals simultaneously. Nonetheless, block codes are widely used, mainly because of their simple generation and parity checking. For improved error correcting properties, there are the convolutional codes that do not require dividing into blocks.